

gen_statem in Depth — A Readable Companion to the OTP Docs

Chapter 13 · checked against OTP 29.1.1 · example tested on OTP 25 · Sep 25, 2026

1. Why a state machine behaviour

Many processes are really state machines written without admitting it: a connection that is connecting, connected or backing off; a payment that is pending, authorised or settled; a device that is idle, busy or faulted. Written as a `gen_server`, the state hides in a `status` field, and the rules are scattered across `case` expressions in every callback. The question “what can happen in this state?” has no single place to look.

`gen_statem` makes the machine explicit. The docs describe it with one line:

```
\text{State} \times \text{Event} \rightarrow \text{Actions}, \backslash \text{State}'
```

and call the result “mostly like an Event-Driven Mealy machine”: outputs (replies, timers, messages) depend on both the current state and the incoming event. For anyone from control systems this is familiar ground. Draw the transition table first, then the code is a direct translation of it.

Chapter 5 introduced `gen_statem` next to `gen_server`. This chapter works through one complete machine, then covers the details that decide whether a real machine is correct.

2. The building blocks

Callback mode

`callback_mode/0` returns one of two modes, optionally with `state_enter`:

Mode	Callbacks	State can be	Good for
<code>state_functions</code>	One function per state: <code>locked(EventType, Event, Data)</code>	Atoms only	Few simple states, one state at a time
<code>handle_event_function</code>	One <code>handle_event(EventType, Event, State, Data)</code>	Any term	Tuple states, shared handling across states
+ <code>state_enter</code>	Also called with <code>(enter, OldState, ...)</code> on every entry	—	Setup that must happen on each entry

State versus data

- **State** is what the machine *is*: the part that decides which events are valid.

- **Data** is what the machine *has*: counters, buffers, sockets, configuration.

The docs give a precise test: if a value changes which events you can handle, it belongs in the state. Postponed events are retried only when the state changes, so a value that should unblock them must be part of the state.

Actions

A callback returns the next state, the new data, and a list of actions:

Action	Effect
{reply, From, Reply}	Answer a <code>call</code> ; the only action executed immediately
postpone	Keep this event; retry it after the next state change
{next_event, Type, Content}	Handle this event next, before anything in the mailbox
{state_timeout, T, Msg}	Timer tied to the current state; cancelled on state change
{{timeout, Name}, T, Msg}	Named timer; survives state changes; cancel explicitly
{timeout, T, Msg}	Event timer; cancelled by any event
hibernate	Shrink memory before waiting (costly; not after every event)

Timers accept `cancel` instead of a time, `update` to change the message without restarting the clock, and `{abs, true}` for absolute deadlines.

Return shortcuts

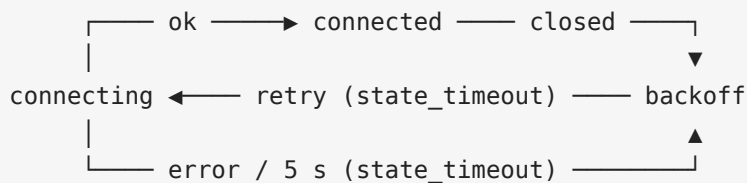
Return	Meaning
{next_state, S, D, Actions}	Go to S (a state change if S $\neq$ current)
{keep_state, D, Actions}	Same state, new data; no enter call
{repeat_state, D, Actions}	Same state, but run the enter call again
keep_state_and_data	Nothing changes

3. A worked example: a reconnecting client

A client that connects to a server, retries with exponential backoff, and makes callers wait while it is not connected. First, the transition table:

State	Event	Actions	Next state
connecting	enter	start async attempt; state_timeout 5 s	—
connecting	attempt succeeded	reset retries	connected

State	Event	Actions	Next state
connecting	attempt failed or 5 s passed	—	backoff
backoff	enter	state_timeout of 100 ms × 2^retries, max 30 s	—
backoff	timeout	—	connecting
connected	send call	reply	connected
connected	socket closed	—	backoff
any other	send call	postpone	unchanged
any	status call	reply with state	unchanged



The code is a direct translation of the table (also in [examples/conn.erl](#)):

```

-module(conn).
-behaviour(gen_statem).
-export([start_link/1, send/2, status/1]).
-export([callback_mode/0, init/1, handle_event/4]).

start_link(ConnectFun) -> gen_statem:start_link(?MODULE, ConnectFun, []).
send(Pid, Msg)          -> gen_statem:call(Pid, {send, Msg}).
status(Pid)             -> gen_statem:call(Pid, status).

callback_mode() -> [handle_event_function, state_enter].

init(ConnectFun) ->
    {ok, connecting, #{connect => ConnectFun, attempt => undefined,
                        retries => 0, sock => undefined}}.

%% --- connecting: one attempt, at most 5 s ---
handle_event(enter, _Old, connecting, #{connect := F} = D) ->
    Self = self(),
    Ref = make_ref(),
    spawn_link(fun() -> Self ! {connect_result, Ref, F()} end),
    {keep_state, D#{attempt := Ref}, [{state_timeout, 5000, give_up}]};
handle_event(info, {connect_result, Ref, {ok, Sock}}, connecting,
    #{attempt := Ref} = D) ->
    {next_state, connected, D#{sock := Sock, retries := 0}};
handle_event(info, {connect_result, Ref, {error, _}}, connecting,
    #{attempt := Ref} = D) ->
    {next_state, backoff, D};
handle_event(state_timeout, give_up, connecting, D) ->

```

```

    {next_state, backoff, D};

%% --- backoff: wait, doubling the delay each time ---
handle_event(enter, _Old, backoff, #{retries := N} = D) ->
    Delay = min(30000, 100 bsl N),
    {keep_state, D#{retries := N + 1}, [{state_timeout, Delay, retry}]};
handle_event(state_timeout, retry, backoff, D) ->
    {next_state, connecting, D};

%% --- connected ---
handle_event(enter, _Old, connected, _D) ->
    keep_state_and_data;
handle_event({call, From}, {send, Msg}, connected, #{sock := S}) ->
    {keep_state_and_data, [{reply, From, {sent, S, Msg}}]};
handle_event(info, {closed, S}, connected, #{sock := S} = D) ->
    {next_state, backoff, D#{sock := undefined}};

%% --- any state ---
handle_event({call, From}, status, State, _D) ->
    {keep_state_and_data, [{reply, From, State}]};
handle_event({call, _From}, {send, _}, _NotConnected, _D) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {connect_result, _StaleRef, _}, _State, _D) ->
    keep_state_and_data.

```

What each feature is doing:

- **State enter** starts every connection attempt in one place, whether we arrived from `init`, from `backoff`, or after a drop. The initial state gets an `enter` call too.
- `state_timeout` bounds each attempt and each backoff. Leaving the state cancels it automatically, so a successful connect never sees a stale `give_up`.
- `postpone` makes `send` wait while disconnected. On entering `connected` the postponed calls are replayed in order, with no queue in the data and no manual bookkeeping.
- **The attempt reference** ties each result to the attempt that produced it (section 4.1 shows why).

In a test where the first two attempts fail, a `send` issued at startup is answered about 300 ms later: two backoffs of 100 ms and 200 ms, then the third attempt succeeds.

4. How the runtime actually does it

The order of a transition

After a callback returns, the engine does roughly this:

1. Sends every `reply` in the order given.
2. If the state changed and `state_enter` is on, calls the `enter` callback.
3. Postpones the current event if asked.
4. On a state change, puts all postponed events back at the front of the queue, oldest first.
5. Inserts `next_event` events ahead of everything else.

6. Starts or cancels timers. A timer of 0 is not started; its event is queued straight away, after any events already queued.

What an enter call may not do

The enter call is not an event, so the docs forbid changing the state, postponing, inserting events or changing the callback module from it. It can update data, set timers and reply. To leave a state immediately after entering it, set `{state_timeout, 0, Msg}` and handle `Msg`.

No default clause

`gen_server` gives you a default `handle_info` that logs and drops unknown messages. `gen_statem` does not: an event that matches no clause raises `function_clause` and crashes the machine.

Never receive inside a callback

The docs are explicit: a catch-all `receive` inside a `gen_statem` callback can swallow system messages “which in turn can lead to unexpected behaviour.” Anything you would wait for with `receive` should be an event handled by the machine, and waiting should be a state.

5. Plausible but wrong

5.1 Accepting a result from an old attempt

```
handle_event(info, {connect_result, {ok, Sock}}, connecting, D) ->
    {next_state, connected, D#{sock := Sock}}.
```

Attempt 1 times out after 5 s, the machine backs off and starts attempt 2, and then attempt 1's late success arrives. The machine is `connecting`, the pattern matches, and it marks itself `connected` on a socket from an attempt it already gave up on. Fix: tag each attempt with a fresh reference, store it in the data, match on it, and drop stale results explicitly, as in the example.

5.2 A catch-all postpone that captures too much

```
handle_event({call, _, _, State, _} when State /= connected ->
    {keep_state_and_data, [postpone]};
handle_event({call, From, status, State, _} ->
    {keep_state_and_data, [{reply, From, State}]}.
```

Clauses are tried in order, so `status` is postponed too. A health check calling `status` hangs for exactly as long as the connection is down, which is precisely when you need the answer. Fix: put specific clauses first, and postpone only the events that really need the connection (`{send, _}` in the example).

5.3 Changing state from an enter call

```
handle_event(enter, _Old, connecting, D) ->
  case quick_check() of
    down -> {next_state, backoff, D};
    up    -> {keep_state, D}
  end.
```

Enter calls may not change the state, so the `down` branch crashes the machine with a bad return. Fix: `{keep_state, D, [{state_timeout, 0, down}]}` and handle `state_timeout, down` like any other event.

5.4 A named timer that outlives its state

```
handle_event(enter, _Old, connected, D) ->
  {keep_state, D, [{timeout, heartbeat}, 1000, ping]};
handle_event({timeout, heartbeat}, ping, connected, D) -> ...
```

Named timeouts survive state changes. The connection drops, the machine goes to `backoff`, the heartbeat fires there, and no clause matches: the machine crashes. Fix: use `state_timeout` when the timer belongs to one state; otherwise cancel it on the way out (`{timeout, heartbeat, cancel}`) or handle it in every state.

5.5 No clause for the unexpected

%% handle_event has clauses only for the messages we expect

A stray message (a late `'DOWN'`, a reply to a timed-out request, a typo) raises `function_clause`, and the machine restarts and loses its data. Unlike `gen_server`, there is no default. Fix: a final clause for `info` events that logs and keeps state. For unexpected calls, reply with an error rather than letting the caller wait.

5.6 Postponing what can never succeed

```
handle_event({call, _}, {send, _}, failed_permanently, _D) ->
  {keep_state_and_data, [postpone]}.
```

The machine never leaves `failed_permanently`, so these calls are never retried. Every caller waits for its full timeout, and the postponed events pile up in the process. Fix: postpone only in states that will eventually be left. Where the answer is known to be “no”, reply `{error, unavailable}` straight away.

5.7 Waiting with receive inside a callback

```
handle_event({call, From}, connect, idle, D) ->
  Sock = start_connect(),
  receive {connected, Sock} -> ok after 5000 -> exit(timeout) end,
  {next_state, connected, D, [{reply, From, ok}]}.
```

For up to 5 seconds the machine cannot answer anything, including system messages from its supervisor and from `sys`. A catch-all variant would swallow them. Fix: the pattern from the example: start the work asynchronously, go to a `connecting` state with a `state_timeout`, and reply when the result event arrives (keep `From` in the data, or postpone the call).

5.8 Data that should be state

```
handle_event({call, _}, {send, _}, connected, #{credit := 0}) ->
    {keep_state_and_data, [postpone]};
handle_event(info, {credit, N}, connected, D) ->
    {keep_state, D#{credit := N}}.
```

Credit arrives but the state is still `connected`, so the postponed sends are never retried (chapter 5, section 4.4). Fix: use a tuple state such as `{connected, Credit}` with `handle_event_function`, or split into `connected` and `blocked`. The docs show exactly this pattern with a complex state.

6. Review checklist and sources

- ☐ The machine has a written transition table, and the code follows it
- ☐ Everything that decides which events are handled lives in the state, not the data
- ☐ Asynchronous results carry a reference to the attempt or request that produced them
- ☐ Specific clauses come before generic `postpone` clauses
- ☐ Enter calls only set up data and timers; immediate transitions use a zero `state_timeout`
- ☐ Per-state timers use `state_timeout`; named timeouts are cancelled or handled in every state
- ☐ A final clause handles unexpected `info` events; unexpected calls get an error reply
- ☐ Events are postponed only in states the machine will eventually leave
- ☐ No `receive` inside callbacks; waiting is modelled as a state

Sources

- `gen_statem` Behaviour, OTP Design Principles: <https://www.erlang.org/doc/system/statem.html>
- `gen_statem` module reference: https://www.erlang.org/doc/apps/stdlib/gen_statem.html